

SI1001 Teoría de la Computación

Verificación de programas

Andrés Sicard Ramírez

Universidad EAFIT

Semestre 2025-1

Esquema de la presentación

- Introducción
- Lógica de Hoare
- FRAMA-C y su plugin WP
- Referencias

- **Introducción**
- Lógica de Hoare
- FRAMA-C y su plugin WP
- Referencias

Cinco preguntas

En el prefacio de [Hei2017] el autor señala que la mayoría de los problemas en Ciencias de la Computación involucran una de las siguientes cinco preguntas:

- (i) *Can the problem be solved by a computer program?*
- (ii) *If not, can you modify the problem so that it can be solved by a program?*
- (iii) *If so, can you write a program to solve the problem?*
- (iv) *Can you convince another person that your program is correct?*
- (v) *Can you convince another person that your program is efficient?*

Clasificación (paradigmas) de los lenguajes de programación

Clasificación

Adaptada de [Scho2015].

- Declarativos (qué)
 - Funcionales (LISP, ML, HASKELL, ...)
 - Lógicos, basados en restricciones (PROLOG, CLP, ...)
- Imperativos (cómo)
 - **von Neumann** (FORTRAN, C, ADA, ...)
 - Orientados a objetos (SMALLTALK, C++, JAVA, ...)
 - *Scripting* (PERL, PYTHON, PHP, ...)

La arquitectura de von Neumann

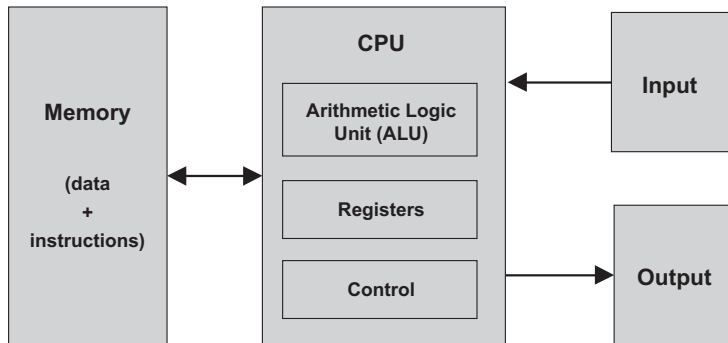


Figura 5.1 en [NS2005]

Programas en lenguajes de máquina

Ejemplo

Programa en lenguaje de máquina que adiciona los números 5 y 6.

```
0010001000000100
0010010000000100
0001011001000010
0011011000000011
1111000000100101
0000000000000101
0000000000000110
0000000000000000
```

Figura 1.2 en [LL2011]

Programas en lenguajes ensambladores

Ejemplo

Programa en lenguaje ensamblador que adiciona los números en las variables **FIRST** y **SECOND** y almacena el resultado en la variable **SUM**.

```
.ORIG x3000      ; Address (in hexadecimal) of the first instruction
LD  R1, FIRST    ; Copy the number in memory location FIRST to register R1
LD  R2, SECOND    ; Copy the number in memory location SECOND to register R2
ADD R3, R2, R1    ; Add the numbers in R1 and R2 and place the sum in
                  ; register R3
ST  R3, SUM       ; Copy the number in R3 to memory location SUM
HALT              ; Halt the program
FIRST .FILL #5    ; Location FIRST contains decimal 5
SECOND .FILL #6   ; Location SECOND contains decimal 6
SUM    .BLKW #1   ; Location SUM (contains 0 by default)
.END             ; End of program
```


Programas en lenguajes imperativos

Ejemplo

Programa en lenguaje C que adiciona los números en las variables `first` y `second` y almacena el resultado en la variable `sum`.

```
int first = 5;  
int second = 6;  
sum = first + second;
```

Programas en lenguajes imperativos

Descripción

Un programa en un lenguaje imperativo es una secuencia de instrucciones que representan comandos:

- instrucciones de asignación
- instrucciones de secuenciación
- instrucciones *if-then-else*
- instrucciones *while*

Razonamiento sobre programas

Categorías

La investigación acerca el razonamiento sobre programas se puede dividir en cuatro categorías [Ach+2010]:

- (i) definir la semántica (nuevos conceptos)
- (ii) razonamiento sobre transformaciones de programas
- (iii) **verificación (formal) de propiedades funcionales** (correspondencia de entrada y salida)
- (iv) verificación (formal) de propiedades no funcionales (consumo de memoria, rendimiento paralelo, etc.)

Verificación de programas

Correctitud parcial y correctitud total

Al establecer la correctitud funcional (correspondencia de entrada y salida) de un programa se distingue entre:

- **Correctitud parcial:** El programa es correcto respecto a su especificación
- **Correctitud total:** Correctitud parcial + terminación

- Introducción
- Lógica de Hoare
- FRAMA-C y su plugin WP
- Referencias

Bibliografía

- Mike Gordon. «Background Reading on Hoare Logic». 2016. URL: <https://www.cl.cam.ac.uk/archive/mjcg/HL/>.

Un pequeño lenguaje de programación imperativo (IMP)

Expresiones aritméticas:

$E ::= N$	(números)
V	(variables)
$E_1 + E_2$	(adición)
$E_1 * E_2$	(multiplicación)

Instrucciones:

$I ::= V := E$	(asignación)
$I_1; I_2$	(secuenciación)
if B then I_1 else I_2	(condicional)
while B do I	(while)

Expresiones booleanas:

$B ::= \text{true} \mid \text{false}$	(constantes)
$E_1 == E_2$	(igualdad)
$E_1 <= E_2$	(menor o igual)

Tripletas de Hoare

Definición

La **tripletas de Hoare** son de la forma

$$\{P\} I \{Q\},$$

donde

- I es un programa en IMP,
- P y Q son condiciones (fórmulas de la lógica de predicados de primer orden con identidad) en las variables del programa,
- P es la **precondición**,
- Q es la **postcondición**.

Tripletas de Hoare

Notación

Para escribir las condiciones P y Q usaremos las siguientes constantes lógicas: $\wedge$ (conjunción), $\vee$ (disyunción), $\supset$ (condicional), $\perp$ (falso) y $\top$ (verdadero).

Tripletas de Hoare

Ejemplo

Para la tripleta de Hoare

$$\{x = 1\} \ x := x + 1 \ \{x = 2\}$$

- P es la precondición que el valor de x es 1,
- Q es la postcondición que el valor de x es 2,
- I es la instrucción de asignación $x := x + 1$.

Especificaciones de correctitud parcial

Definición

Una tripleta de Hoare $\{P\} I \{Q\}$ es una **especificación de correctitud parcial**.

Especificaciones de correctitud parcial

Definición

Una tripleta de Hoare $\{P\} I \{Q\}$ es una **especificación de correctitud parcial**.

Semántica

La especificación de correctitud parcial $\{P\} I \{Q\}$ es **verdadera** sii cuando el programa I es ejecutado en un estado que satisface la precondition P y si la ejecución del programa I termina, entonces el estado en el cual el programa I termina satisface la postcondición Q .

Especificaciones de correctitud parcial

Ejemplos

- $\{x = 1\} \ x := x + 1 \ \{x = 2\}$ es verdadera.
- $\{x = 1\} \ x := x + 1 \ \{x = 3\}$ es falsa.

Especificaciones de correctitud parcial

Observación

$\{P\} I \{Q\}$ es una especificación de correctitud **parcial** porque **no** es necesario que el programa I termine para que la especificación sea verdadera. Solo es necesario que **si** el programa I termina entonces la poscondición Q sea satisfecha.

Especificaciones de correctitud parcial

Ejemplos

- $\{x = 1\}$ while true $x := 42$ $\{x = 2\}$ es verdadera.
- $\{\perp\} I \{Q\}$ es verdadera para todo I y Q .
- $\{P\} I \{\top\}$ es verdadera para todo I y P .

Lógica de Hoare y especificaciones de correctitud parcial

Descripción

La lógica de Hoare es un sistema formal deductivo para la tripletas de Hoare $\{P\} I \{Q\}$.

Lógica de Hoare

Axioma de asignación

$$\frac{}{\vdash \{Q[x \mapsto E]\} \ x \ := \ E \ \{Q\},}$$

donde $Q[x \mapsto E]$ denota el resultado de substituir cada ocurrencia libre de la variable x por la expresión E en la condición Q .

Lógica de Hoare

Axioma de asignación

$$\frac{}{\vdash \{Q[x \mapsto E]\} x := E \{Q\},}$$

donde $Q[x \mapsto E]$ denota el resultado de substituir cada ocurrencia libre de la variable x por la expresión E en la condición Q .

Ejemplos (instancias del axioma de asignación)

- $\vdash \{y = 2\} x := 2 \{y = x\}.$

Axioma de asignación

$$\frac{}{\vdash \{Q[x \mapsto E]\} x := E \{Q\},}$$

donde $Q[x \mapsto E]$ denota el resultado de substituir cada ocurrencia libre de la variable x por la expresión E en la condición Q .

Ejemplos (instancias del axioma de asignación)

- $\vdash \{y = 2\} x := 2 \{y = x\}.$
- $\vdash \{y = x\} z := y \{z = x\}.$

Regla de inferencia: fortalecimiento de una precondition

$$\frac{\vdash P \supset P' \quad \vdash \{P'\} I \{Q\}}{\vdash \{P\} I \{Q\}}.$$

Lógica de Hoare

Regla de inferencia: fortalecimiento de una precondition

$$\frac{\vdash P \supset P' \quad \vdash \{P'\} I \{Q\}}{\vdash \{P\} I \{Q\}}.$$

Regla de inferencia: debilitamiento de una postcondición

$$\frac{\vdash \{P\} I \{Q'\} \quad \vdash Q' \supset Q}{\vdash \{P\} I \{Q\}}.$$

Ejemplo (demostración formal)

Demostremos que $\vdash \{r = x\} \text{ q } := 0 \{r = x + (y \times q)\}.$

Lógica de Hoare

Ejemplo (demostración formal)

Demostremos que $\vdash \{r = x\} \text{ q } := 0 \{r = x + (y \times q)\}$.

Demostración.

1. $\vdash r = x \supset r = x \wedge 0 = 0$ (por lógica)
2. $\vdash \{r = x \wedge 0 = 0\} \text{ q } := 0 \{r = x \wedge q = 0\}$ (axioma de asignación)
3. $\vdash \{r = x\} \text{ q } := 0 \{r = x \wedge q = 0\}$ (fortalecimiento de precondición en 1 y 2)
4. $\vdash r = x \wedge q = 0 \supset r = x + (y \times q)$ (por aritmética)
5. $\vdash \{r = x\} \text{ q } := 0 \{r = x + (y \times q)\}$ (debilitamiento de postcondición en 3 y 4)

Regla de inferencia: secuenciación

$$\frac{\vdash \{P\} I_1 \{Q\} \quad \vdash \{Q\} I_2 \{R\}}{\vdash \{P\} I_1 ; I_2 \{R\} .}$$

- Introducción
- Lógica de Hoare
- FRAMA-C y su plugin WP
- Referencias

Preliminares

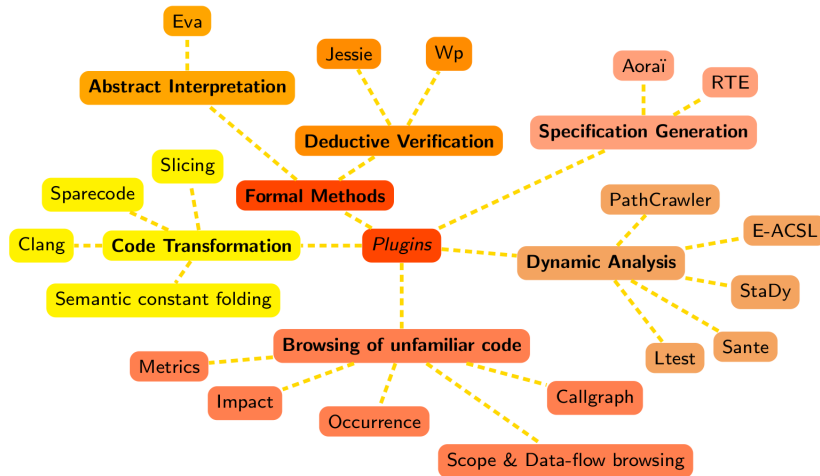
- La bibliografía para esta sección es [Bla2024].
- La numeración (de secciones, definiciones, teoremas, figuras, páginas, etc.) en esta sección corresponde a la numeración en [Bla2024].
- Versiones de las herramientas usadas para los ejemplos de esta sección:
 - FRAMA-C 30.0 (Zinc).
 - Compilador para el lenguaje C: GCC 12.2.0.

FRAMA-C y su plugin WP

Descripción

- FRAMA-C (*FRAmework for Modular Analysis of C code*) es una plataforma para análisis de programas escritos en C creada por la CEA LIST y el Inria.
- ACSL (*ANSI/ISO C Specification Language*) (pronunciado «Axel») es el lenguaje de especificación para ANSI/ISO C usado por FRAMA-C.
- WP (*Weakest Precondition*) es un plugin para realizar verificación formal de programas en FRAMA-C.

FRAMA-C y su plugin WP



(Figura [pág. 13])

Descripción

«The goal of a **function contract** is to state the properties of the input that are expected by the function, and in exchange the properties that will be assured for the output.» [pág. 22]

Los contratos están compuestos de **pre-condiciones** y **post-condiciones**, las cuales se escriben en ACSL.

Ejemplo

Calcular el valor absoluto de un número entero.

```
1  int abs(int val) {  
2      if (val < 0) return -val;  
3      return val;  
4  }
```

(continua en la próxima diapositiva)

Ejemplo (continuación)

Adicionamos una primera post-condición escrita en ACSL.

```
1  /*@
2    ensures \result >= 0;
3  */
4  int abs(int val) {
5    if (val < 0) return -val;
6    return val;
7  }
```

(continua en la próxima diapositiva)

Ejemplo (continuación)

Añadimos una segunda post-condición escrita en ACSL.

```
1  /*@
2    ensures \result >= 0;
3    ensures (val >= 0 ==> \result == val) && (val < 0 ==> \result == -val);
4  */
5  int abs(int val) {
6    if (val < 0) return -val;
7    return val;
8  }
```

(continúa en la próxima diapositiva)

FRAMA-C y su plugin WP

Ejemplo (continuación)

Después de nombrar el programa **abs.c** ejecutamos la instrucción

```
$ frama-c-gui abs.c
```

y demostramos las post-condiciones utilizando el plugin WP.

(continua en la próxima diapositiva)

Ejemplo (continuación)

¿Es nuestro programa libre de errores? ¿Qué acerca de errores en tiempo de ejecución?

Ejemplo (continuación)

¿Es nuestro programa libre de errores? ¿Qué acerca de errores en tiempo de ejecución?

El plugin RTE (*runtime errors*) adiciona anotaciones al programa para evitar errores en tiempo de ejecución (*integer overflow*, *invalid pointer dereferencing*, división por cero, etc.)

(continua en la próxima diapositiva)

Ejemplo (continuación)

La anotación adicionada por el plugin RTE es debido a la representación de números en el computador. Como es señalado por *The GNU C Library Reference Manual* para la versión 2.36:

Most computers use a two's complement integer representation, in which the absolute value of `INT_MIN` (the smallest possible `int`) cannot be represented; thus, `abs (INT_MIN)` is not defined.

(continua en la próxima diapositiva)

Ejemplo (continuación)

Si un número entero es representado con n bits usando la representación de complemento a dos los valores que se pueden representar están en el intervalo $[-2^{n-1}, 2^{n-1} - 1]$

En mi computador, un `int` de C es representado con 32 bits, es decir, los valores que se pueden representar son $[-2^{31}, 2^{31} - 1] = [-2147483648, 2147483647]$.

(continua en la próxima diapositiva)

Ejemplo (continuación)

Programa que muestra el problema empleando la función `abs` de la biblioteca de C de GNU (*GNU C library*).

```
1  #include <limits.h>
2  #include <stdio.h>
3  #include <stdlib.h>
4
5  int main() {
6      printf("Maximum integer (INT_MAX): %d\n", INT_MAX);
7      printf("Minimum integer (INT_MIN): %d\n", INT_MIN);
8      printf("Absolute value of INT_MIN: %d (error)\n", abs(INT_MIN));
9  }
```

(continua en la próxima diapositiva)

FRAMA-C y su plugin WP

Ejemplo (continuación)

Después de nombrar el programa anterior `glibc-abs.c`, compilar y ejecutar el programa con las siguientes instrucciones:

```
$ gcc -Wall -Wextra glibc-abs.c -o glibc-abs  
$ ./glibc-abs  
Maximum integer (INT_MAX): 2147483647  
Minimum integer (INT_MIN): -2147483648  
Absolute value of INT_MIN: -2147483648 (error)
```

(continua en la próxima diapositiva)

FRAMA-C y su plugin WP

Ejemplo (continuación)

Adicionamos la pre-condición asociada a la representación de los números enteros.

```
1  #include <limits.h>
2
3  /*@
4     requires INT_MIN < val;
5
6     ensures \result >= 0;
7     ensures (val >= 0 ==> \result == val) && (val < 0 ==> \result == -val);
8  */
9  int abs(int val) {
10     if (val < 0) return -val;
11     return val;
12 }
```


FRAMA-C y su plugin WP

Ejemplo (continuación)

Llamamos FRAMA-C con la instrucción

```
$ frama-c-gui -wp -rte abs.c
```

Tema

- Introducción
- Lógica de Hoare
- FRAMA-C y su plugin WP
- Referencias

Referencias

- [Ach+2010] Peter Achten, Marko van Eekelen, Pieter Koopam y Marco T. Morazán. «Trends in *Trends in Functional Programming* 1999/2000 versus 2007/2008». En: *Higher-Order Symbolic Computation* 23.4 (2010), págs. 465-487. DOI: [10.1007/s10990-011-9074-z](https://doi.org/10.1007/s10990-011-9074-z) (vid. pág. 11).
- [Bla2024] Allan Blanchard. «Introduction to C program proof with Frama-C and its WP plugin». Recent version from the GitHub repository. 2 de nov. de 2024. URL: https://github.com/AllanBlanchard/tutoriel_wp (vid. pág. 34).
- [Gor2016] Mike Gordon. «Background Reading on Hoare Logic». 2016. URL: <https://www.cl.cam.ac.uk/archive/mjcg/HL/> (vid. pág. 14).
- [Hei2017] James L. Hein. *Discrete Structures, Logic, and Computability*. 4.^a ed. Jones & Bartlett Learning, 2017 (vid. pág. 4).
- [LL2011] Kenneth C. Loudon y Kenneth A. Lambert. *Programming Languages. Principles and Practice*. 3.^a ed. Cengage Learning, 2011 (vid. págs. 7, 8).

Referencias

- [NS2005] Noam Nisan y Schocken Shimon. *The Elements of Computing Systems. Building a Modern Computer from First Principles*. MIT Press, 2005 (vid. pág. 6).
- [Sco2015] Michael L. Scott. *Programming Language Pragmatics*. 4.^a ed. Morgan Kaufmann, 2015 (vid. pág. 5).